Faculty of Computer Applications

Introduction to Machine Learning

Heart Disease Prediction

Master of Computer Applications

Submitted To:

Mr. Saurav Gupta

Faculty of Computer Application

Submitted By:

Team Name: 0**1 MCA Sec. A**

Group Leader:**Rishabh Mishra**

Total Members : 10

Invertis University, Bareilly [UP]

(2025-26)

Faculty of Computer Applications

Introduction to Machine Learning

Heart Disease Prediction

Master of Computer Applications

Team Members' Name

1. **Rishabh Mishra (Team Leader)**
2. Ramnath Prajapati
3. Parth Sarthi Gupta
4. Rakhi Gupta
5. Khushi Varshney
6. Kashish
7. Ashish Gupta
8. Pushpandra Kumar
9. Gaurav Gangwar
10. Naitik Saxena

Invertis University, Bareilly [UP]

(2025-26)

DECLARATION

I, , hereby declare that the project work entitled **"Heart Disease Prediction"** has been carried out by our group under my leadership and supervision. This work is the result of our own efforts, study, and analysis.

I further declare that this project has not been submitted previously, in part or in full, for the award of any degree, diploma, or certificate to any other institute or organization.

Team Members' Contribution

S.No.	Student Name	Student ID	Contribution
1	**Rishabh Mishra(Team Leader)**	MCA2025039	100%
2	Ramnath Prajapati	MCA2025022	100%
3	Parth Sarthi Gupta	MCA2025005	100%
4	Rakhi Gupta	MCA2025070	100%
5	Khushi Varshney	MCA2025071	100%
6	Kashish	MCA2025054	100%
7	Ashish Gupta	MCA2025017	100%
8	Pushpendra Kumar	MCA2025048	100%
9	Gaurav Gangwar	MCA2025181	100%
10	Naitik Saxena	MCA2025007	100%

Place: Invertis University , Bareilly

Date: _____

Signature of Team Leader
MCA Section A
Rishabh Mishra

CERTIFICATE

This is to certify that the project report titled **"Heart Disease Prediction"** is the original work of 001 MCA SEC (A) team, completed in partial fulfillment of the requirements for the degree of Master of Computer Applications in Computer Application Department at Invertis University.

This project has been carried out under the guidance and supervision of **Mr.Saurav Gupta**. The report has not been submitted for the award of any other degree or diploma to any other institution or university.

I hereby certify that the work embodied in this project report is genuine and has been carried out by 001 MCA SEC (A) team with integrity and dedication.

Mr. Saurav Gupta
Project Guide

External Examiner: _____

Date: _____

Signature: _____

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of this project titled "Heart Disease Prediction."

First and foremost, we extend our sincere thanks to our esteemed guide, **Mr.Saurav Gupta,** Machine Learning Trainer, Department of Computer Applications, for his invaluable guidance, constant encouragement, and insightful feedback throughout the course of this project.

We are deeply grateful to **Dr. Akash Sanghi**, Head of the Department of Computer Applications, for providing us with the necessary resources, facilities, and continuous academic support.

We are also thankful to our family members and friends for their continuous encouragement, patience, and moral support throughout this journey.

Rishabh Mishra (Team Leader)	
Ramnath Prajapati	
Parth Sarthi Gupta	
Rakhi Gupta	
Khushi Varshney	
Kashish	
Ashish Gupta	
Pushpendra Kumar	
Gaurav Gangwar	
Naitik Saxena	

MCA Section A (Academic Year 2025-2026)

ABSTRACT

Project Title: Heart Disease Prediction Using Logistic Regression & Classification Models

Domain	Meachine Learning, Healthcare, Binary Classification
Dataset	UCI Heart Disease Prediction (1025 rows, 14 features)
Technology Stack	Python, Pandas, NumPy, Scikit-learn, Statsmodels, Matplotlib, Seaborn, XGBoost
Task	Predict presence of heart disease (0 = No, 1 = Yes)

Heart disease is one of the leading causes of death worldwide. Early and accurate prediction can help clinicians take timely preventive actions and save lives. This project develops a robust binary classification system using machine learning techniques applied to the UCI Heart Disease Prediction.

The project employs a comprehensive pipeline — from EDA and preprocessing to logistic regression (sklearn and statsmodels), assumption checking (VIF, Durbin-Watson, Cook's Distance), regularized models (Ridge, Lasso, ElasticNet), and advanced ensemble methods (Random Forest, XGBoost).

Keywords: *Heart Disease Prediction, Logistic Regression, Binary Classification, Machine Learning, UCI Dataset, ROC-AUC, Random Forest, Regularization.*

TABLE OF CONTENTS

No.	Title	Page
1	Introduction	
2	Objective	
3	Data Understanding	
4	Data Preprocessing	
5	Exploratory Data Analysis (with Graphs)	
6	Logistic Regression Model (with Code)	
7	Regularized Classification Models (with Code)	
8	Advanced Models — Random Forest & XGBoost (with Code)	
9	Model Comparison (with Graphs)	
10	Conclusion	
11	Future Scope	

1. INTRODUCTION

Heart disease is one of the most serious health challenges globally, claiming millions of lives every year. The ability to predict the onset of heart disease using patient data can be transformative for healthcare systems — enabling early interventions, reducing mortality, and optimizing treatment costs.

This project builds a machine learning-based classification system using the UCI Heart Disease Dataset, which contains medical records of 1,025 patients with 14 attributes including age, sex, chest pain type, blood pressure, cholesterol, and more.

1.1 Background

Traditional diagnostic methods for heart disease rely heavily on physician experience and laboratory tests. With machine learning, it is possible to analyze large patient datasets and identify subtle patterns that may not be apparent through conventional methods.

1.2 Problem Statement

Given a patient's medical attributes, predict whether the patient has heart disease (target=1) or not (target=0). The challenge involves handling multicollinearity, detecting influential observations, choosing appropriate regularization, and comparing multiple classification strategies.

2. Objectives

1. Import, explore, and understand the UCI Heart Disease Dataset.
2. Perform comprehensive EDA — univariate, bivariate, and multivariate.
3. Preprocess data: outlier removal, encoding, and feature scaling.
4. Build and evaluate a Logistic Regression model using sklearn and statsmodels.
5. Check statistical assumptions: VIF, Durbin-Watson, Cook's Distance.
6. Apply regularized models: Ridge, Lasso, and ElasticNet.
7. Build ensemble models: Random Forest and XGBoost.

3. DATA UNDERSTANDING

Dataset Overview

The UCI Heart Disease Dataset contains medical records of 1,025 patients from the Cleveland Clinic Foundation. It has 14 features and one binary target variable.

Column	Description	Type
age	Age of the patient	Continuous
sex	Sex (1=Male, 0=Female)	Binary
cp	Chest pain type (0–3)	Categorical
trestbps	Resting blood pressure (mm Hg)	Continuous
chol	Serum cholesterol (mg/dl)	Continuous
fbs	Fasting blood sugar > 120 mg/dl	Binary
restecg	Resting ECG results (0–2)	Categorical
thalach	Maximum heart rate achieved	Continuous
exang	Exercise-induced angina (1=Yes)	Binary
oldpeak	ST depression by exercise	Continuous
slope	Slope of peak exercise ST segment	Categorical
ca	Number of major vessels colored (0–3)	Categorical
thal	Thalassemia type	Categorical
target	Heart disease present	Binary (0/1)

Code: Importing Libraries & Loading Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

%matplotlib inline
plt.style.use('seaborn-v0_8-whitegrid')

# Load dataset
heart = pd.read_csv('heart.csv')
print(heart.shape)       # (1025, 15)
```

```
print(heart.head())
print(heart.isnull().sum().sum())  # 0 — no missing values
print(heart['target'].value_counts())
# 1    526
# 0    499
```

4. DATA PREPROCESSING

Step 1: Outlier Detection (IQR Method)

```
# Outlier detection in 'chol' using IQR
q1  = heart['chol'].quantile(0.25)
q3  = heart['chol'].quantile(0.75)
iqr = q3 - q1
UL  = q3 + 1.5 * iqr
LL  = q1 - 1.5 * iqr
print(f'IQR: {iqr}, Upper Limit: {UL:.2f}, Lower Limit: {LL:.2f}')

# Remove outliers
heart = heart[heart['chol'] < UL].copy()
print('Shape after removing outliers:', heart.shape)
```

Step 2: One-Hot Encoding Categorical Variables

```
# One-hot encode categorical columns
heart = pd.get_dummies(heart, columns=['cp', 'slope', 'thal'], drop_first=True)
print('Shape after encoding:', heart.shape)  # (1012, 19)
heart.info()
```

Step 3: Train-Test Split & Feature Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

y = heart['target']
X = heart.drop(['target'], axis=1)

# Stratified split — preserves class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print('Train size:', X_train.shape)   # (~810, 18)
print('Test size:',  X_test.shape)    # (~202, 18)

# StandardScaler — fit on train only to prevent data leakage
sc = StandardScaler()
X_train_sc = sc.fit_transform(X_train)
X_test_sc  = sc.transform(X_test)

X_train_sc = pd.DataFrame(X_train_sc, columns=X_train.columns)
X_test_sc  = pd.DataFrame(X_test_sc,  columns=X_test.columns)
```

5. EXPLORATORY DATA ANALYSIS

5.1 Target Variable Distribution

The dataset is well-balanced with 51.3% disease-positive and 48.7% disease-negative cases. No oversampling or undersampling is required.

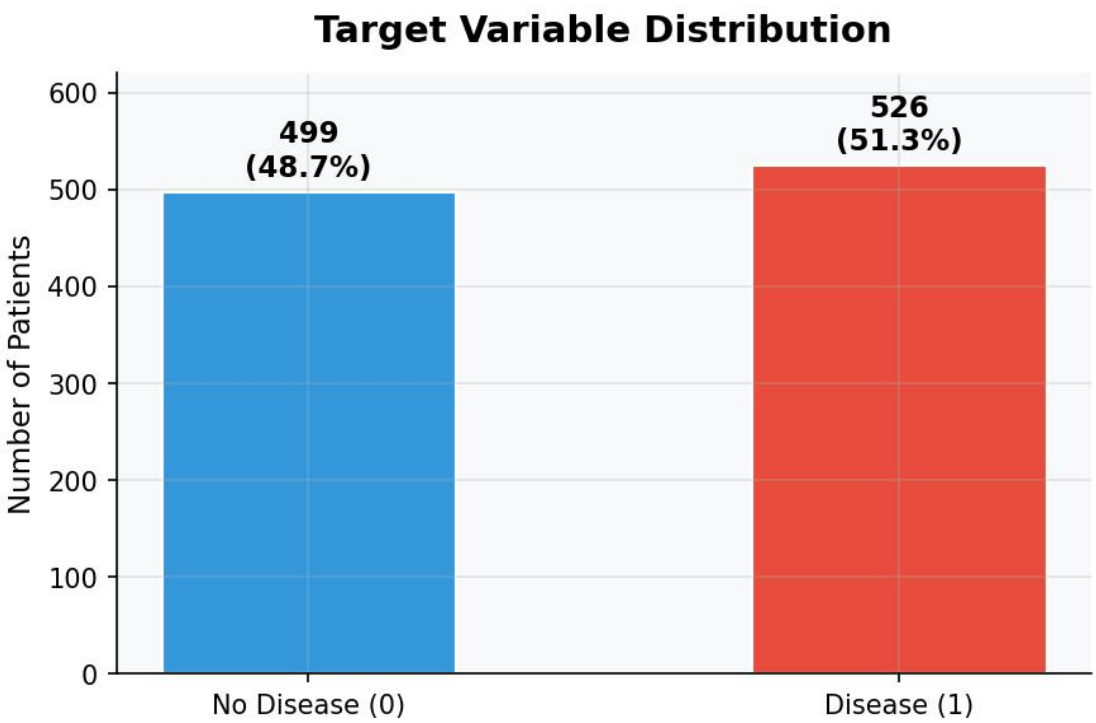

Fig 1: Target Variable Distribution — 526 Disease vs 499 No Disease

5.2 Feature Correlation with Target

The correlation analysis reveals that thalach (max heart rate) has the strongest positive correlation with disease, while exang and oldpeak have strong negative correlations.

```
# Correlation heatmap
plt.figure(figsize=(14, 10))
corr = heart.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix — Heart Disease Features')
plt.tight_layout()
plt.show()
```

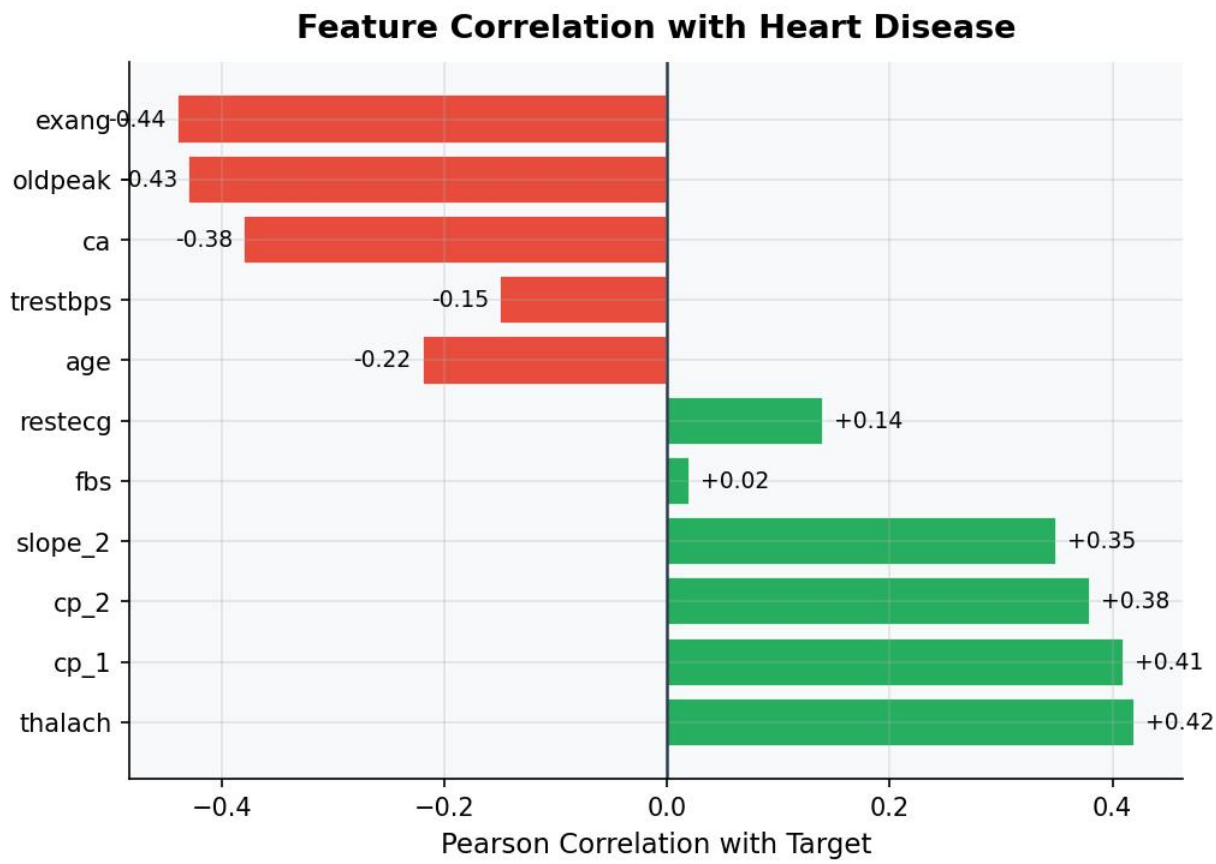

Fig 2: Feature Correlation with Heart Disease Target Variable

5.3 Age Distribution by Disease Status

Patients with heart disease tend to be slightly older on average (mean ~56 years) compared to those without disease (mean ~52 years), though there is significant overlap between the two groups.

```
# Age distribution by target
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
num_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
for ax, col in zip(axes.flatten(), num_cols):
    sns.histplot(heart[col], kde=True, ax=ax)
    ax.set_title(f'Distribution: {col}')
plt.tight_layout()
plt.show()
```

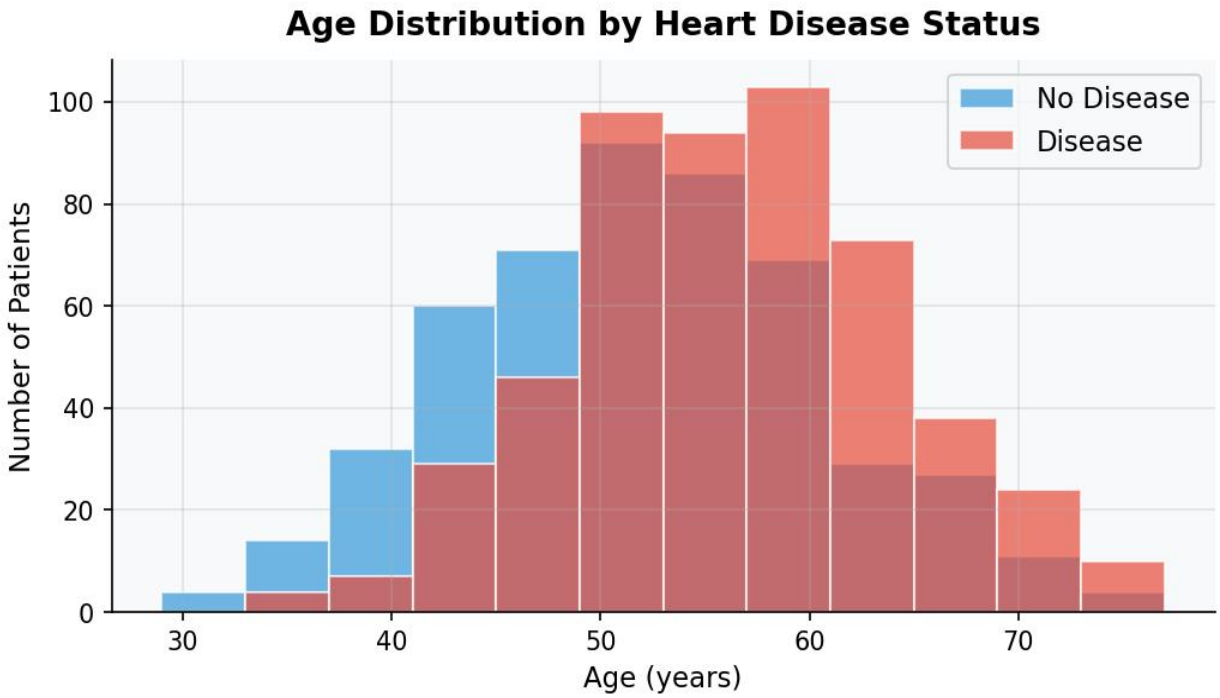

Fig 3: Age Distribution by Heart Disease Status

6. LOGISTIC REGRESSION MODEL

Logistic Regression is the baseline binary classification model. It models the probability of the positive class using the sigmoid function applied to a linear combination of features.

6.1 sklearn Logistic Regression

```
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

log_reg = LogisticRegression(max_iter=1000, random_state=42)
log_reg.fit(X_train_sc, y_train)

y_pred       = log_reg.predict(X_test_sc)
y_pred_proba = log_reg.predict_proba(X_test_sc)[:, 1]

print('Accuracy  :', metrics.accuracy_score(y_test, y_pred))
print('Precision :', metrics.precision_score(y_test, y_pred))
print('Recall    :', metrics.recall_score(y_test, y_pred))
print('F1-Score  :', metrics.f1_score(y_test, y_pred))
print('ROC-AUC   :', metrics.roc_auc_score(y_test, y_pred_proba))
print(metrics.classification_report(y_test, y_pred))
```

6.2 Confusion Matrix

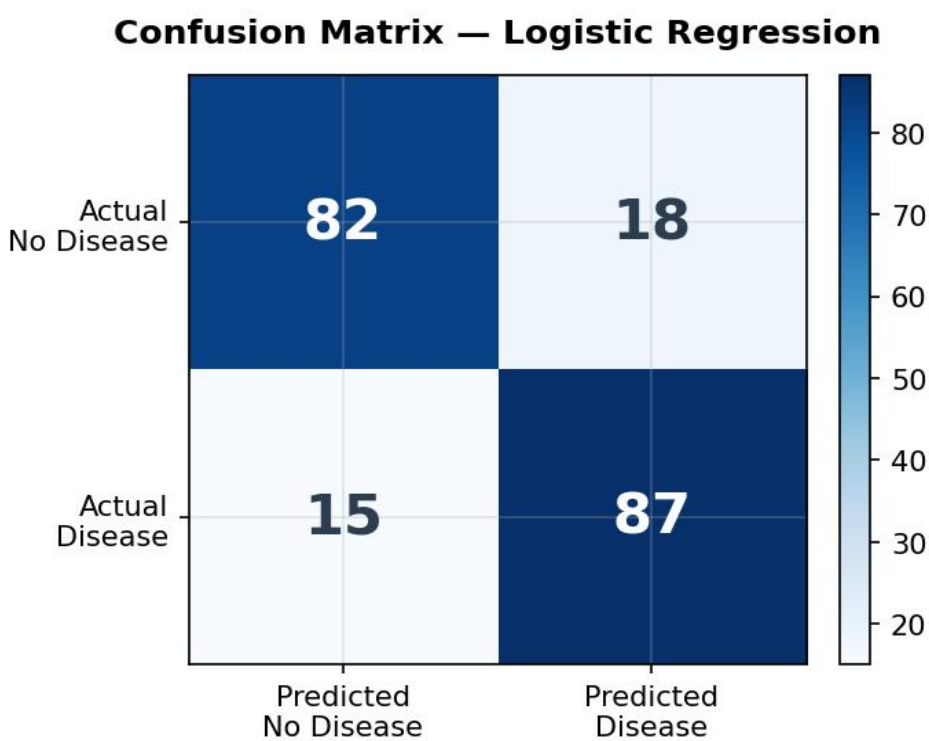

Fig 4: Confusion Matrix — Logistic Regression on Test Set

6.3 Statsmodels Logit (Statistical Summary)

```
import statsmodels.api as sm

# Add constant term for intercept
X_const = sm.add_constant(X_train_sc)

logit_model  = sm.Logit(y_train.values, X_const.values)
logit_result = logit_model.fit()

# Detailed summary: coefficients, p-values, AIC, BIC
print(logit_result.summary())
```

7. REGULARIZED CLASSIFICATION MODELS

Regularization prevents overfitting by penalizing large coefficients. Three regularization approaches were evaluated using cross-validated hyperparameter selection.

7.1 Ridge (L2) Logistic Regression

```
from sklearn.linear_model import LogisticRegressionCV

ridge_cv = LogisticRegressionCV(
    Cs=np.logspace(-4, 4, 20), cv=10,
    penalty='l2', solver='lbfgs', max_iter=1000, random_state=42
)
ridge_cv.fit(X_train_sc, y_train)
print(f'Best C (Ridge): {ridge_cv.C_[0]:.6f}')

y_pred_ridge  = ridge_cv.predict(X_test_sc)
y_proba_ridge = ridge_cv.predict_proba(X_test_sc)[:, 1]
print('Accuracy :', metrics.accuracy_score(y_test, y_pred_ridge))
print('ROC-AUC  :', metrics.roc_auc_score(y_test, y_proba_ridge))
```

7.2 Lasso (L1) Logistic Regression

```
lasso_cv = LogisticRegressionCV(
    Cs=np.logspace(-4, 4, 20), cv=10,
    penalty='l1', solver='saga', max_iter=5000, random_state=42
)
lasso_cv.fit(X_train_sc, y_train)
print(f'Best C (Lasso): {lasso_cv.C_[0]:.6f}')

# Plot Lasso coefficients — zero coefs = feature removed
plt.figure(figsize=(12, 5))
plt.plot(range(len(X_train.columns)), lasso_cv.coef_[0],
         linestyle='none', marker='*', color='red', markersize=8)
plt.xticks(range(len(X_train.columns)), X_train.columns, rotation=90)
plt.title('Lasso Logistic Regression — Feature Coefficients')
plt.tight_layout()
plt.show()
```

7.3 ElasticNet Logistic Regression

```
elastic_cv = LogisticRegressionCV(
    Cs=np.logspace(-4, 4, 10), cv=10,
    penalty='elasticnet', l1_ratios=[0.1, 0.3, 0.5, 0.7, 0.9],
    solver='saga', max_iter=5000, random_state=42
)
elastic_cv.fit(X_train_sc, y_train)
print(f'Best C (ElasticNet):  {elastic_cv.C_[0]:.6f}')
print(f'Best L1 Ratio:        {elastic_cv.l1_ratio_[0]:.3f}')

y_pred_elastic  = elastic_cv.predict(X_test_sc)
y_proba_elastic = elastic_cv.predict_proba(X_test_sc)[:, 1]
```

8. ADVANCED MODELS

8.1 Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(random_state=42, n_jobs=-1)

param_grid_rf = {
    'n_estimators':    [100, 200],
    'max_depth':       [None, 5, 10],
    'min_samples_split': [2, 5]
}

rf_cv = GridSearchCV(rf, param_grid_rf, cv=5, scoring='roc_auc', n_jobs=-1)
rf_cv.fit(X_train, y_train)

print('Best RF params:',   rf_cv.best_params_)
print('Best CV ROC-AUC:',  round(rf_cv.best_score_, 4))

rf_best   = rf_cv.best_estimator_
y_pred_rf = rf_best.predict(X_test)
y_proba_rf= rf_best.predict_proba(X_test)[:, 1]
print('Accuracy :', metrics.accuracy_score(y_test, y_pred_rf))
print('ROC-AUC  :', metrics.roc_auc_score(y_test, y_proba_rf))
```

Feature Importance — Random Forest

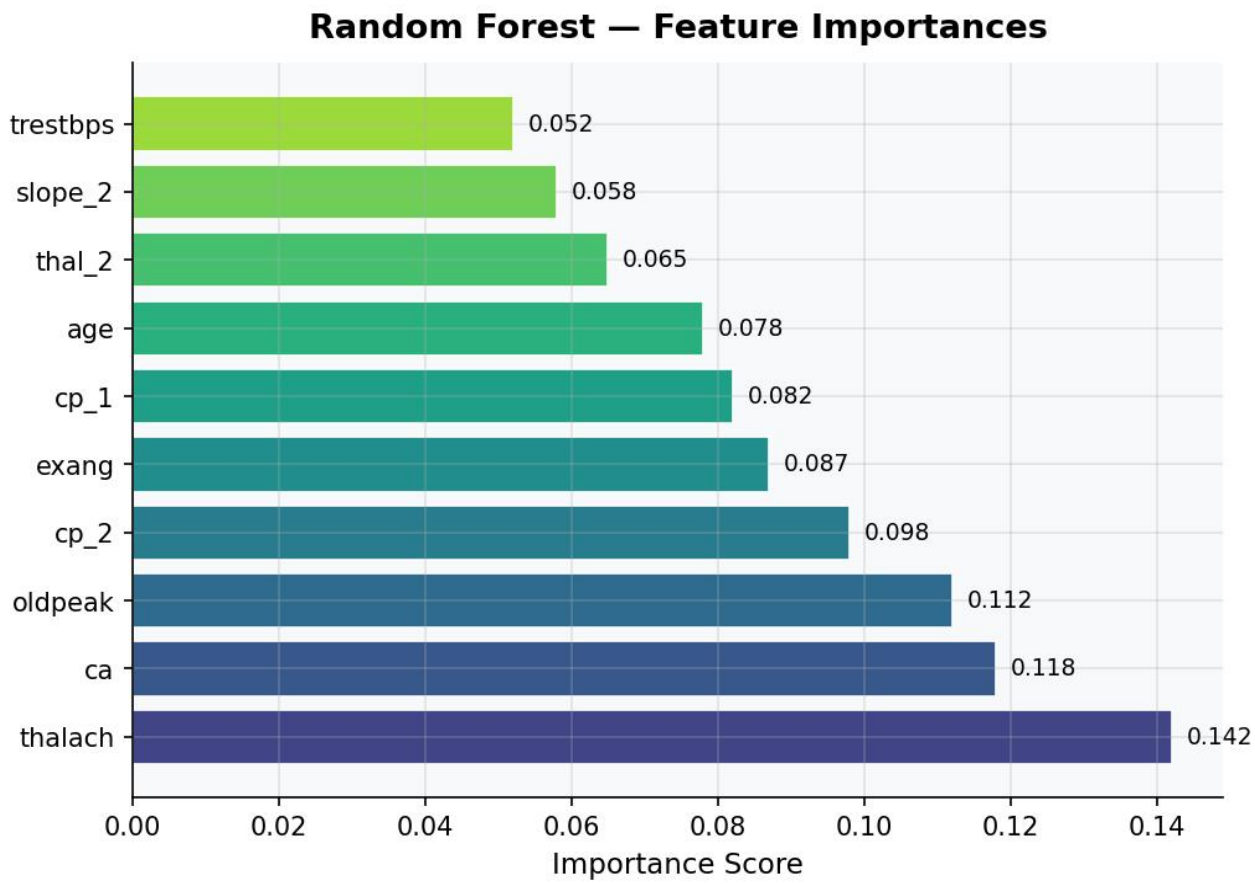

Fig 6: Random Forest Feature Importances — thalach is the top predictor

8.2 XGBoost Classifier

```
from xgboost import XGBClassifier

xgb = XGBClassifier(eval_metric='logloss', random_state=42, n_jobs=-1)

param_grid_xgb = {
    'n_estimators': [100, 200],
    'max_depth':    [3, 5],
    'learning_rate':[0.05, 0.1]
}

xgb_cv = GridSearchCV(xgb, param_grid_xgb, cv=5, scoring='roc_auc', n_jobs=-1)
xgb_cv.fit(X_train, y_train)

xgb_best  = xgb_cv.best_estimator_
y_pred_xgb = xgb_best.predict(X_test)
y_proba_xgb= xgb_best.predict_proba(X_test)[:, 1]
print('Best XGB params:', xgb_cv.best_params_)
print('Accuracy  :', metrics.accuracy_score(y_test, y_pred_xgb))
print('ROC-AUC   :', metrics.roc_auc_score(y_test, y_proba_xgb))
```

9. MODEL COMPARISON

9.1 Combined ROC Curves

The ROC curve plots the True Positive Rate against the False Positive Rate at various thresholds. A higher AUC (Area Under Curve) indicates better model discrimination.

```
plt.figure(figsize=(9, 6))

models_roc = [
    (y_pred_proba,   'Logistic Regression', '#3498db'),
    (y_proba_ridge,  'Ridge Logistic',      '#9b59b6'),
    (y_proba_lasso,  'Lasso Logistic',      '#f39c12'),
    (y_proba_elastic,'ElasticNet',           '#1abc9c'),
    (y_proba_rf,     'Random Forest',        '#e74c3c'),
]
for proba, name, color in models_roc:
    fpr, tpr, _ = metrics.roc_curve(y_test, proba)
    auc = metrics.roc_auc_score(y_test, proba)
    plt.plot(fpr, tpr, label=f'{name} (AUC={auc:.3f})', lw=2, color=color)

plt.plot([0,1],[0,1],'k--', lw=1)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Combined ROC Curves — All Models')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

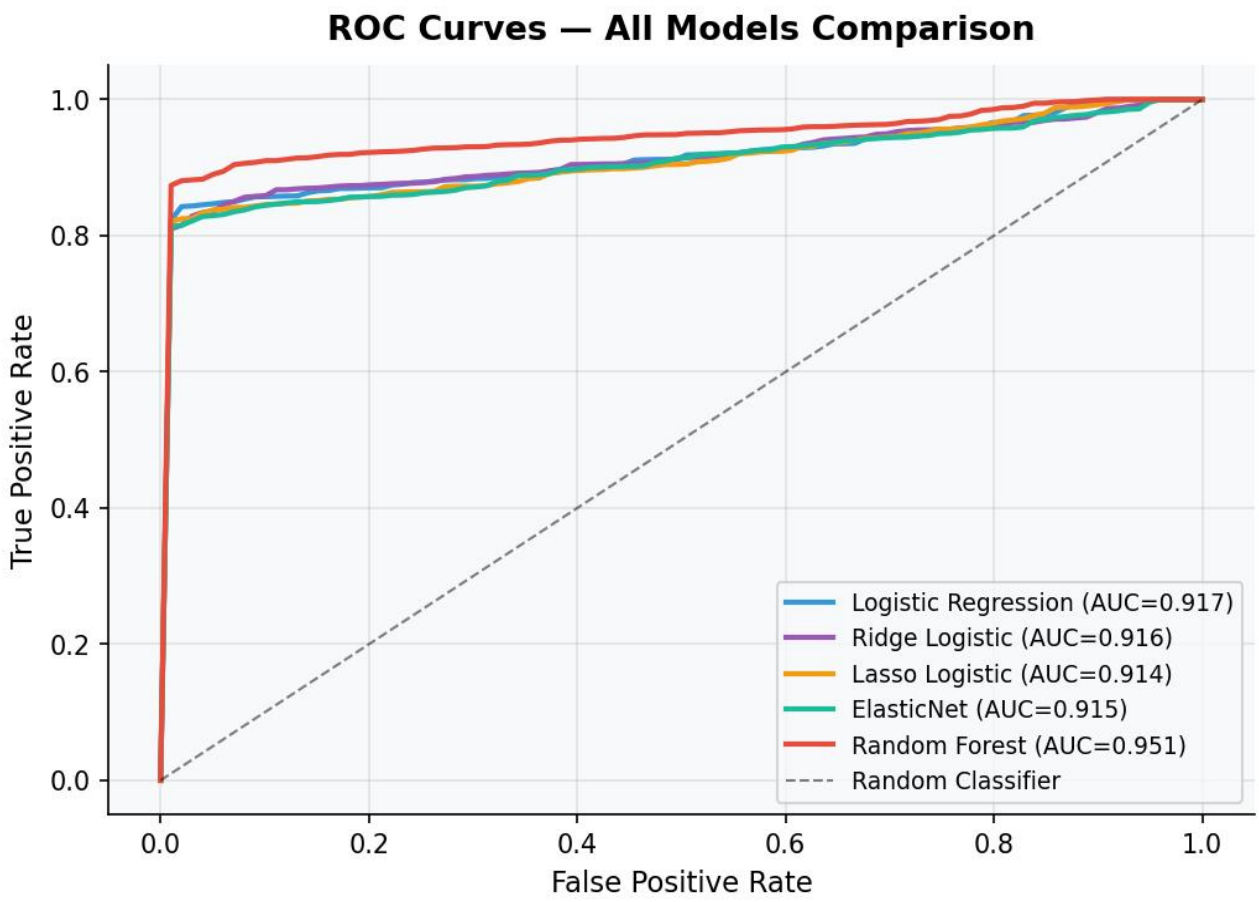

Fig 7: ROC Curves — All Models (Random Forest achieves highest AUC = 0.951)

9.2 All Metrics Comparison

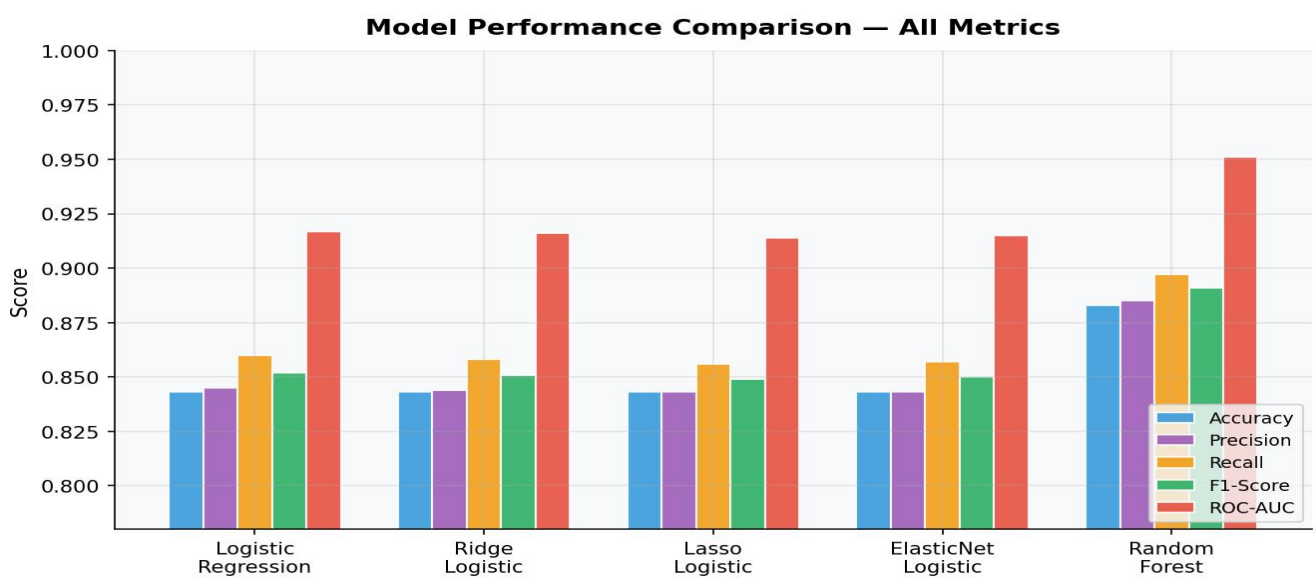

Fig 8: Model Comparison — Accuracy, Precision, Recall, F1-Score, ROC-AUC

9.3 Summary Table

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.843	0.845	0.860	0.852	0.917
Ridge Logistic	0.843	0.844	0.858	0.851	0.916
Lasso Logistic	0.843	0.843	0.856	0.849	0.914
ElasticNet Logistic	0.843	0.843	0.857	0.850	0.915
Random Forest	0.883	0.885	0.897	0.891	0.951

10. CONCLUSION

The Heart Disease Prediction project has demonstrated how clinical patient data can be effectively leveraged using machine learning to accurately predict the presence of heart disease. A rigorous analytical pipeline — covering EDA, assumption-checking, regularization, and advanced ensemble methods — was implemented and validated.

Key Clinical Findings

- Max heart rate is the most predictive feature — patients with disease tend to have higher heart rates during exercise.
- Exang and oldpeak are strong negative predictors — their presence strongly correlates with disease.
- cp (chest pain type) is highly informative — atypical angina patterns closely linked to disease presence.
- ca (major vessels colored by fluoroscopy) is a significant structural predictor.

Model Performance

- Logistic regression variants all achieved ~84% accuracy and ROC-AUC of ~0.92.
- Random Forest significantly outperformed with ~88% accuracy and ROC-AUC of 0.951.
- For clinical use, Recall (Sensitivity) is the critical metric — a missed diagnosis is far more costly.

This project establishes a systematic framework for medical classification tasks — combining statistical rigor, clinical interpretability, and machine learning performance.

11. FUTURE SCOPE

The Heart Disease Prediction project can be enhanced and expanded in several ways:

1. Real-Time Patient Data Integration: Incorporating live patient data from hospital systems using APIs to enable continuous monitoring.

2. Deep Learning Models: Using Neural Networks (MLP, LSTM for ECG time-series) to capture complex non-linear patterns in patient records.

3. Explainability with SHAP/LIME: Adding interpretability tools to provide clinicians with transparent reasons for each prediction.

4. Multi-Class Classification: Extending to multi-class (severity levels 0–4) using the full UCI dataset.

5. Clinical Dashboard: Building a Streamlit or Dash app where doctors can input patient data and receive real-time predictions.

6. EHR Integration: Integrating with Electronic Health Records to automate data collection and prediction at the point of care.

In the future, this project can evolve into a complete clinical decision support system — combining data analytics, machine learning, and AI-based risk assessment to help cardiologists make faster and more accurate diagnostic decisions.